ClinicBook — Complete PRD Pack (Single File)

What this is: The full PRD pack for **ClinicBook** (working title only — the user can rename it later) — a clinic booking app for Pakistan — merged into one file. It follows the same shape as the earlier LeadLoop PRD pack.

The idea in one line: Small clinics in one Pakistani city take appointments by phone only, and patients travel just to ask for a time. ClinicBook lets a patient pick a doctor, pick a time slot, and book — and get the confirmation on WhatsApp.

Built as a learning experiment first. No user interviews were done before writing this pack. The assumptions below are marked, and the last section lists the questions to answer if this ever graduates from experiment to real product.

Honest positioning (read this before building): Big competitors already exist in Pakistan (oladoc, marham). ClinicBook does NOT try to beat them nationwide. The wedge is: **small clinics in ONE city, WhatsApp-first, Urdu-friendly, and free for clinics to start.** V1 launches in Charsadda only — but the product is designed to be usable and sellable worldwide later: every city carries its own country and timezone, so a new city or country is added as a data row, never as a redesign. Nothing beyond Charsadda is built in v1.

How to use with an AI coding tool (Cursor, Codex, Claude, DeepSeek, ...):

1. Paste this whole file and say: *"Read PART 1 (AGENTS) first, then build Phase 0 of PART 5 (IMPLEMENTATION PLAN). Do not skip ahead."*
2. After each phase: *"Run the acceptance tests for this phase from PART 6 and fix failures before moving on."*

Labels used: [Decision] = already made, don't re-debate. **[Assumption]** = treated as true for the experiment, not validated. **[Out of scope]** = not built in v1.

Decisions already made for you: Stack = Next.js 16 (App Router) + TypeScript strict + Tailwind + Supabase (Postgres + Auth) + WhatsApp Cloud API (free tier) + Vercel (free tier). One city in v1: Charsadda. Patients book without an account (name + phone only). Clinics log in with email + password. Timezone for everything: Asia/Karachi. No payments in v1 — patients pay cash at the clinic. Design: mobile-first, Urdu + English side by side, teal primary.

Read order inside this file: PART 1 AGENTS → PART 2 PRD → PART 3 TECHNICAL SPEC → PART 4 DESIGN SPEC → PART 5 IMPLEMENTATION PLAN → PART 6 ACCEPTANCE TESTS → PART 7 VERIFICATION LESSON + SETUP.

PART 1 — AGENTS.md (read this first)

AGENTS.md — Instructions for coding agents building ClinicBook

> Read this file first. It outranks any default behavior. The other parts are: `PRD` (what), `TECHNICAL SPEC` (how), `DESIGN SPEC` (look), `IMPLEMENTATION PLAN` (order), `ACCEPTANCE TESTS` (done = passing).

1. How to work

1. **Read order:** this part → `IMPLEMENTATION PLAN` → the spec sections for the current phase only. Do not implement future phases early.
2. **One phase at a time.** Finish a phase fully (code + tests + acceptance checks) before starting the next. Say which phase you completed and what remains.
3. **Small steps:** implement one route/component at a time; run `tsc --noEmit` and the relevant tests after each step.
4. **Ask before:** adding a dependency, changing the schema, or deviating from any **[Decision]** in the specs. Everything else, decide yourself and note it.

2. Code standards (non-negotiable)

- TypeScript `strict: true`. **No** `any` — use `unknown` + narrowing if stuck.
- Validate **every** API input with the shared zod schemas (client validation is UX sugar, never the boundary).
- Never import server-only modules (service-role client, WhatsApp secrets) from client components. If unsure, grep your imports.
- One component per file; co-locate tiny helpers; no barrel-file gymnastics.
- **Tests for every logic flow and UI:** unit tests (Vitest) for validations, slot generation, phone normalization; Playwright for the critical paths only (browse, booking, clinic confirm/cancel).
- Fix the intentional defect: `TECHNICAL SPEC` §3 contains one deliberately wrong SQL line, clearly marked. Correct it in the migration.

3. What NOT to do

- Do not "improve" the product (no new features, no redesigns). Build the spec.
- Do not refactor working code to chase elegance mid-phase.

- Do not hard-delete user data; cancel appointments instead of deleting them (per PRD FR-008).
- Do not commit secrets. Use `.env.local` (git-ignored); document every new variable in `TECHNICAL SPEC` §7.
- Do not add payments, video calls, lab reports, or prescriptions. They are **[Out of scope]** for v1, no matter how easy they look.

4. When the spec is ambiguous

1. Re-read the three spec parts — the answer is usually there.
2. If truly ambiguous: choose the **simplest** option, implement it, and add a one-line `// ASSUMPTION:` comment plus a note in your phase summary.
3. Never silently invent product behavior that contradicts the PRD.

5. Definition of "phase complete"

- `tsc --noEmit` passes, all tests pass, `npm run build` succeeds.
- Every acceptance test for the phase (in `ACCEPTANCE TESTS`) passes — quote the results.
- No `TODO` / `FIXME` left in phase code except those explicitly deferred by the plan.

PART 2 — PRD.md

ClinicBook — Product Requirements Document (v1)

Version: 1.0 (experiment) · **Date:** 2026-10-26 · **Status:** Ready for build

1. Product overview and vision

ClinicBook is a simple web app for booking doctor appointments in one city.

Vision: No patient should travel to a clinic just to ask, "Doctor kab milenge?" (When will the doctor be available?) — the answer should be on their phone.

One-line pitch: Pick a doctor, pick a time, get the confirmation on WhatsApp. Built for small clinics in Charsadda that today take appointments by phone only.

2. Problem statement

Small clinics in Pakistan's smaller cities work like this today:

- A patient hears about a doctor, travels to the clinic, and asks for a time — or calls repeatedly and gets a busy line.
- The clinic keeps appointments in a paper register or in memory. Double bookings happen. Nobody reminds the patient.
- Big booking platforms (oladoc, marham) focus on big cities and big hospitals. A small clinic in Charsadda is not on them, and would not know how to join.

Proposed solution: One simple website for one city. Patients browse clinics and doctors, see real available time slots, and book in under a minute. The clinic sees today's appointments on one screen, confirms or cancels with one tap, and the patient gets the confirmation on WhatsApp — the app most Pakistanis already use every day.

3. Target audience and personas

Primary persona — Fatima, patient, Charsadda

- 32, mother of two, uses WhatsApp daily, reads simple Urdu better than English. Her phone is her only internet device.
- Pain: "My son had fever. I went to the clinic at 10am and the doctor was only coming at 4pm. I had to go back twice."
- Needs: see the doctor's real timings, book a slot, and get a WhatsApp message she can show at the clinic.

Secondary persona — Dr. Ahmed, small clinic owner, Charsadda

- Runs a 2-room clinic with one assistant. Appointments live in a paper register. He has a smartphone and WhatsApp, but has never used any software for his clinic.
- Pain: "Patients call all day asking for timings. My assistant writes names in a register and sometimes two patients get the same time."
- Needs: one screen with today's appointments, one tap to confirm or cancel, and a way to set his working hours once.

[Assumption]: Fatima and Dr. Ahmed represent the target users well enough for the experiment. No real user interviews were conducted.

Non-users (v1): big hospitals, labs, pharmacies, patients outside Charsadda, anyone needing video consultations or online payment.

4. Goals, objectives, success metrics

Experiment goal: A working v1 built by an AI coding tool from this pack, passing all acceptance tests.

Product metrics (defined so the app can measure them later):

- % of bookings confirmed by the clinic within 2 hours (target: >80%)
- Double bookings (target: 0 — enforced by the database, see TECHNICAL SPEC)
- Bookings completed per clinic per week (target: trending up)
- % of patients who receive their WhatsApp confirmation (target: >95%)

5. Scope

5.1 v1 — in scope

1. Public browse: list of clinics in Charsadda; search by clinic/doctor name; filter by specialty
2. Clinic page: clinic info + its doctors
3. Doctor page: doctor info, consultation fee (display only, cash at clinic), working days, available time slots
4. Booking flow: pick a date, pick an available slot, enter name + phone (Pakistan mobile format), optional note, confirm — no account needed
5. WhatsApp messages: booking-received, appointment-confirmed, appointment-cancelled (templates in English + simple Urdu)
6. Clinic accounts: signup/login with email + password (Supabase Auth)
7. Clinic dashboard: today's appointments, confirm/cancel, filter by doctor/date
8. Clinic management: clinic profile (name, city, address, phone), doctors (name, specialty, fee), weekly schedules (days, hours, slot length)
9. World-ready model: every clinic belongs to a city, and every city belongs to a country with its own timezone; v1 seeds Charsadda only, and any new city or country in the world is added later as data rows without a redesign
10. Mobile-first, Urdu + English bilingual UI

5.2 Out of scope (v1) — do not build

- Payments of any kind (online, card, JazzCash/Easypaisa) — patients pay cash at the clinic **[Out of scope]**
- Video consultations **[Out of scope]**
- Lab reports, prescriptions, medical records **[Out of scope]**
- Patient accounts, patient login, online cancellation by patients (patients cancel by calling/WhatsApping the clinic) **[Out of scope]**
- Clinics outside Charsadda in v1 (the world-ready data model supports other cities and countries later as data rows) **[Out of scope]**

- Multi-clinic accounts (one account = one clinic in v1) **[Out of scope]**
- SMS notifications (WhatsApp only in v1) **[Out of scope]**
- WhatsApp inbound webhook / replies handling (outbound messages only) **[Out of scope]**
- Admin panel, ratings/reviews, doctor search nationwide **[Out of scope]**
- Mobile apps (responsive web only) **[Out of scope]**

6. User roles and permissions (v1)

Role	Scope
Patient (no account)	Browse clinics/doctors/slots; create a booking with name + phone
Clinic staff (one account per clinic)	Full access to their own clinic only: dashboard, confirm/cancel, doctors, schedules, clinic profile

[Decision]: Exactly one account per clinic in v1. Every clinic row belongs to one owner user; staff can only ever see their own clinic's data (enforced by database Row Level Security — see TECHNICAL SPEC).

[Decision]: Patients do NOT get accounts in v1. Booking with name + phone keeps the flow under one minute for a first-time user on a phone. A patient identifies their booking by the WhatsApp confirmation they received.

7. Functional requirements

FR-001 — Browse clinics (public)

User story: As a patient, I can see all clinics in Charsadda, so I can pick one near me. **Acceptance criteria:**

- Clinics list shows: clinic name, area/address, phone, number of doctors.
- Search matches clinic name and doctor name (case-insensitive, partial).
- Specialty filter narrows the list; filter + search combine.
- Only Charsadda clinics show in v1; the city is shown on the page, not hidden.

FR-002 — Clinic page

User story: As a patient, I can see a clinic's doctors, so I can choose who to see. **Acceptance criteria:**

- Shows clinic name, address, phone, and its doctors with specialty + fee.
- Each doctor links to their booking page.

- A clinic with no doctors shows a helpful empty state, not a blank page.

FR-003 — Doctor page + slot picker

User story: As a patient, I can see a doctor's real available times, so I don't travel for nothing. **Acceptance criteria:**

- Shows doctor name, specialty, qualifications (if given), fee (PKR, display only — "pay cash at clinic" is stated on the page).
- Date picker offers the next 14 days only.
- Slots come from the doctor's weekly schedule, minus already-booked slots (pending + confirmed block a slot; cancelled frees it).
- Past slots on today's date are hidden.
- A day with no schedule shows "No clinic today" in English + Urdu.

FR-004 — Book an appointment (no account)

User story: As a patient, I can book in under a minute with just my name and phone number. **Fields:** patient name (required), phone (required, Pakistan mobile format `03XX-XXXXXXX`), note (optional, e.g. "fever since yesterday"). **Acceptance criteria:**

- Booking with only name + valid phone succeeds.
- Invalid phone is rejected with a clear message ("Please enter a valid mobile number like 03XX-XXXXXXX").
- Booking a slot that just got taken by someone else fails gracefully with "Sorry, this slot was just booked — please pick another time" (never a crash, never a double booking).
- Success screen shows: doctor, clinic, date, time, and "Confirmation sent on WhatsApp" — in English + Urdu.

FR-005 — WhatsApp confirmations

User story: As a patient, I get my booking confirmation on WhatsApp, so I have proof to show at the clinic. **Acceptance criteria:**

- On booking: "booking received" WhatsApp message is sent to the patient's number (template: English + simple Urdu, see TECHNICAL SPEC §6).
- On clinic confirm: "appointment confirmed" WhatsApp message is sent.
- On clinic cancel: "appointment cancelled" WhatsApp message is sent.
- Every message attempt is recorded (sent / failed + reason) in a message log the clinic staff can see.
- If WhatsApp is not configured (dev mode), booking still succeeds and the message is recorded as "not sent (WhatsApp not configured)" — booking must never fail because WhatsApp failed.

FR-006 — Clinic signup / login / logout

User story: As clinic staff, I can create an account and log in securely, so only my clinic's data is visible to me. **Acceptance criteria:**

- Signup with email + password (min 8 chars) works; verification email sent.
- Wrong password shows a generic error (no "user not found" hint).
- Logout ends the session; protected routes redirect to `/login`.

FR-007 — Clinic dashboard (today's appointments)

User story: As clinic staff, I open the app and instantly see who is coming today. **Acceptance criteria:**

- Default view = today's appointments for the whole clinic, newest first by time; filter by doctor and by date.
- Each row: patient name, phone (tap-to-call link), doctor, time, status.
- One tap confirms a pending appointment; one tap cancels (with a reason field, optional). Status changes trigger the matching WhatsApp message.
- Counts shown: pending / confirmed / cancelled for the selected day.
- Empty day shows a helpful empty state, not a blank page.

FR-008 — Manage clinic profile, doctors, schedules

User story: As clinic staff, I set up my clinic once and update it when things change. **Acceptance criteria:**

- Clinic profile: name (required), city (Charsadda, fixed in v1), address (required), phone (required, Pakistan format).
- Doctor: name (required), specialty (required, from a fixed list), qualifications (optional), fee in PKR (optional, display only).
- Schedule per doctor: for each weekday, working or closed; start time, end time, slot length in minutes (15 / 30 / 60 — default 30).
- Changing a schedule never deletes existing appointments; it only affects future slot generation.
- Deleting a doctor is not allowed if they have future appointments — the doctor is deactivated instead (keeps history intact).

FR-009 — Cancel, never delete

User story: As clinic staff, I can cancel an appointment without losing its record. **Acceptance criteria:**

- Cancelled appointments stay visible in the day view with status "cancelled" and the reason (if given).
- Nothing is ever hard-deleted by users in v1.

8. Non-functional requirements

- **NFR-001 Performance:** Browse + booking pages render in <2s on 4G.
- **NFR-002 Security:** All data access enforced by Supabase Row Level Security; no client-side filtering as a security boundary. Booking creation goes through a server route (service-role client, server only) so anonymous users can book without reading other patients' data. See TECHNICAL SPEC.
- **NFR-003 Reliability:** No double booking under any timing — enforced by a database unique index, not just by UI checks.
- **NFR-004 Usability:** Booking completable by a non-technical user with no training, on a 360px phone, in under a minute.
- **NFR-005 Language:** Every patient-facing label appears in English + simple Urdu (see DESIGN SPEC). Urdu must be simple everyday wording, not formal or hard Urdu.
- **NFR-006 Maintainability:** TypeScript strict mode, no `any`; every business rule (slots, phone validation, status changes) has a unit test.
- **NFR-007 Compatibility:** Modern browsers (Chrome, Edge, Firefox, Safari — last 2 versions); mobile-first responsive down to 360px width.
- **NFR-008 Cost:** $0. Only free tiers: Supabase free, Vercel free, WhatsApp Cloud API free test tier. No paid service may be added without asking first.

9. User flows

Flow A — Patient books (the core flow)

1. Patient opens the site → sees Charsadda clinics → searches "children" (or taps the Children specialty filter).
2. Taps a clinic → taps a doctor → picks tomorrow → picks an available slot (e.g. 4:30pm).
3. Enters name + phone → confirms → success screen with full details.
4. WhatsApp "booking received" message arrives. Later, when the clinic confirms, a WhatsApp "appointment confirmed" message arrives.

Flow B — Clinic confirms the day's list

1. Staff logs in → dashboard shows today's appointments.
2. Taps Confirm on each pending booking → patient gets WhatsApp confirmation automatically.

3. A patient who can't come: staff taps Cancel → patient gets WhatsApp cancellation automatically.

Flow C — Clinic sets up

1. Staff signs up → creates clinic profile (Charsadda, address, phone).
2. Adds doctors + specialties + fees.
3. Sets each doctor's weekly schedule (days, hours, 30-minute slots).
4. Slots appear for patients immediately — no extra step.

10. Assumptions and open questions

[Assumption]: Patients are happy to book without an account if WhatsApp confirms it. (No interviews done — validate before any real launch.) **[Assumption]:** Clinics will list themselves for free if setup takes under 10 minutes. (No interviews done.) **[Assumption]:** WhatsApp Cloud API free test tier (limited recipient numbers) is enough for the experiment. **Open questions (for a real product, not this experiment):** Would clinics pay later, and how much? Would patients trust a new site over a phone call? How would clinics be onboarded — who visits them? Deliberately unanswered here.

PART 3 — TECHNICAL_SPEC.md

ClinicBook — Technical Specification (v1)

1. Stack **[Decision]**

Layer	Choice	Why (one line)
Framework	Next.js 16, App Router	One language end-to-end; AI tools know it best
Language	TypeScript, `strict: true`, no `any`	Catches bugs before runtime
Styling	Tailwind CSS	Fast, consistent utility styling
Database + Auth	Supabase (Postgres + Auth)	No backend ops; Row Level Security built in

Layer	Choice	Why (one line)
WhatsApp	WhatsApp Cloud API (Meta, free tier)	Patients already use WhatsApp daily
Hosting	Vercel (free tier)	Zero-config deploy
Validation	zod (shared client/server schemas)	One source of truth for input rules
Forms	React Hook Form + zod resolver	Accessible, tested form handling
Tests	Vitest (unit) + Playwright (critical e2e only)	Fast feedback; e2e only for browse + booking + clinic flows

Do not add a state manager (React state + server components suffice), a component library (hand-rolled per DESIGN SPEC), or an ORM (Supabase client + SQL migrations).

2. Architecture

Monolith-first. Next.js App Router:

```
app/
  (public)/page.tsx              # Charsadda clinics list (home)
  (public)/clinics/[id]/page.tsx # clinic page + doctors
  (public)/doctors/[id]/page.tsx # doctor page + slot picker + booking form
  (public)/booked/page.tsx       # booking success screen
  (clinic)/login/page.tsx        # public
  (clinic)/signup/page.tsx       # public
  (clinic)/dashboard/page.tsx    # today's appointments (protected)
  (clinic)/doctors/page.tsx      # manage doctors + schedules (protected)
  (clinic)/settings/page.tsx     # clinic profile (protected)
  api/
    clinics/route.ts             # GET public list (search/filter)
    doctors/[id]/slots/route.ts  # GET available slots for a date
    appointments/route.ts        # POST booking (server route, service-role)
    appointments/[id]/route.ts   # PATCH confirm/cancel (clinic JWT)
lib/
  supabase/ (client, server, admin)
  validations/ (zod schemas)
  slots/ (slot generation + availability)
  whatsapp/ (message sender: WhatsApp driver + log driver)
```

- **Public reads:** clinics, doctors, schedules, and slot availability are readable with the anon key under RLS SELECT policies (they are public listings, not private data).
- **Booking creation:** `POST /api/appointments` is a server route. It validates with zod, checks slot availability inside a transaction, inserts with the service-role client (server only), and

queues the WhatsApp message. Anonymous users never get direct database write access.

- **Clinic reads/writes:** Server Components + Route Handlers using the clinic user's JWT; RLS enforces "own clinic only". Never trust client-side filtering.
- **No webhook in v1:** the app only sends WhatsApp messages; it does not receive replies. **[Out of scope]**

3. Data model

```
-- Cities: v1 seeds Charsadda only. Any new city or country in the world
-- is added later as a row, never as a redesign. Every city carries its
-- own country and timezone so the same app works worldwide.
create table cities (
  id uuid primary key default gen_random_uuid(),
  name text not null unique,
  slug text not null unique,   -- e.g. 'charsadda', later 'peshawar', 'london'
  country text not null default 'Pakistan',   -- e.g. 'Pakistan', later any country
  timezone text not null default 'Asia/Karachi',   -- IANA timezone per city
  active boolean not null default true
);

-- One row per clinic; exactly one owner user in v1.
create table clinics (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null references auth.users(id) on delete cascade,
  city_id uuid not null references cities(id),
  name text not null check (char_length(name) between 1 and 120),
  address text not null,
  phone text not null check (char_length(phone) <= 20),
  created_at timestamptz not null default now(),
  unique(owner_id)
);

create type specialty as enum
  ('general','children','women','skin','heart','bones','eyes','dental',
   'ent','mental','other');

create table doctors (
  id uuid primary key default gen_random_uuid(),
  clinic_id uuid not null references clinics(id) on delete cascade,
  name text not null check (char_length(name) between 1 and 120),
  specialty specialty not null,
  qualifications text,
  fee_pkr integer check (fee_pkr is null or fee_pkr >= 0),
  active boolean not null default true,
  created_at timestamptz not null default now()
);
create index doctors_clinic_idx on doctors(clinic_id) where active;

-- Weekly schedule: one row per weekday a doctor works.
create table doctor_schedules (
  id uuid primary key default gen_random_uuid(),
```

```
  doctor_id uuid not null references doctors(id) on delete cascade,
  weekday smallint not null check (weekday between 0 and 6),
  -- 0 = Sunday ... 6 = Saturday
  start_time time not null,
  end_time time not null,
  slot_minutes smallint not null default 30
    check (slot_minutes in (15, 30, 60)),
  check (end_time > start_time),
  unique(doctor_id, weekday)
);

create type appointment_status as enum
  ('pending','confirmed','cancelled','completed');

create table appointments (
  id uuid primary key default gen_random_uuid(),
  doctor_id uuid not null references doctors(id) on delete cascade,
  clinic_id uuid not null references clinics(id) on delete cascade,
  patient_name text not null check (char_length(patient_name) between 1 and 120),
  patient_phone text not null check (char_length(patient_phone) <= 20),
  appointment_date date not null,
  start_time time not null,
  end_time time not null,
  status appointment_status not null default 'pending',
  note text check (char_length(note) <= 500),
  cancel_reason text check (char_length(cancel_reason) <= 300),
  created_at timestamptz not null default now()
);
-- THE rule that makes double booking impossible: the same doctor, date
-- and start time can only have ONE non-cancelled appointment.
create unique index appointments_no_double_book
  on appointments(doctor_id, appointment_date, start_time)
  where status in ('pending','confirmed');
create index appointments_clinic_day_idx
  on appointments(clinic_id, appointment_date, start_time);
```

> **INTENTIONAL DEFECT — verification lesson:** the index line below is deliberately wrong. The builder must find it and fix it in the migration. Do not copy it as-is.

```
create index appointments_doctor_date_idx on clinics(doctor_id, appointment_date);
-- ^ WRONG on purpose: clinics has no doctor_id or appointment_date columns.
-- Correct it to:
-- create index appointments_doctor_date_idx
--   on appointments(doctor_id, appointment_date, start_time);
```

```
-- Every WhatsApp attempt, so failures are visible instead of silent.
create table message_log (
  id uuid primary key default gen_random_uuid(),
  appointment_id uuid not null references appointments(id) on delete cascade,
  clinic_id uuid not null references clinics(id) on delete cascade,
  template text not null,  -- 'booking_received' | 'confirmed' | 'cancelled'
```

```
  recipient_phone text not null,
  status text not null,    -- 'sent' | 'failed' | 'not_configured'
  error_text text,
  created_at timestamptz not null default now()
);
create index message_log_appt_idx on message_log(appointment_id, created_at desc);
```

Row Level Security (required, not optional)

- Enable RLS on `cities`, `clinics`, `doctors`, `doctor_schedules`, `appointments`, `message_log`.
- Public SELECT: anyone (anon) may read `cities` (active only), `clinics`, `doctors` (active only), `doctor_schedules`. Nobody anonymous may read `appointments` or `message_log`.
- Clinic staff: a user may SELECT/UPDATE/INSERT/DELETE a row iff its `clinic_id` belongs to a clinic whose `owner_id = auth.uid()`. For `clinics` itself, iff `owner_id = auth.uid()`. Apply to every table.
- The booking route bypasses RLS via the service-role client — that client must never be importable from client components.

4. API routes

Method	Route	Auth	Purpose
GET	`/api/clinics?search=&specialty=`	Public	List Charsadda clinics (doctors embedded)
GET	`/api/doctors/[id]/slots?date=YYYY-MM-DD`	Public	Available slots for one doctor + date
POST	`/api/appointments`	Server route	Create booking (zod + availability + insert + WhatsApp)
GET	`/api/clinic/appointments?date=&doctor_id=`	Clinic JWT	List own clinic's appointments
PATCH	`/api/appointments/[id]`	Clinic JWT	Confirm / cancel / complete own appointment
GET/PATCH	`/api/clinic/profile`	Clinic JWT	Get / update own clinic profile
GET/POST	`/api/clinic/doctors`	Clinic JWT	List / add own doctors

Method	Route	Auth	Purpose
PATCH	`/api/clinic/doctors/[id]`	Clinic JWT	Edit / deactivate own doctor
GET/PUT	`/api/clinic/doctors/[id]/schedule`	Clinic JWT	Get / replace own doctor's weekly schedule

- All request bodies validated with the shared zod schemas; invalid → 400 with field-level errors.
- Booking a taken slot → 409 with the message from PRD FR-004 ("slot was just booked"), never a 500.
- Status change via PATCH must also write a `message_log` row (via the WhatsApp sender) in the same request flow.

5. Slot generation rules **[Decision]**

1. Take the doctor's schedule row for the requested weekday. No row → no slots ("No clinic today").
2. Slice `[start_time, end_time)` into `slot_minutes` chunks.
3. Remove slots already taken by appointments with status `pending` or `confirmed` on that date.
4. If the date is today, remove slots whose start time has already passed in Asia/Karachi time.
5. Offer dates from today through today + 13 days (14 days total).
6. All time math uses Asia/Karachi explicitly — never the server's local timezone. (This exact bug appeared in a sibling project: a greeting computed in server UTC said "Good morning" at 1pm PKT. Do not repeat it.)

6. WhatsApp messages (templates)

Templates must be submitted for approval in Meta's dashboard before real sending works (user-side step, see PART 7). Texts below are the exact wording to submit — English first, then simple Urdu.

booking_received (sent on booking):

> Your appointment request at {clinic_name} with Dr. {doctor_name} on {date} at {time} has been received. The clinic will confirm shortly. Aapki appointment ki darkhwast {clinic_name} mein Dr. {doctor_name} ke paas {date} ko {time} bajay mil gayi hai. Clinic jald confirm karega.

confirmed (sent on clinic confirm):

> Your appointment at {clinic_name} with Dr. {doctor_name} on {date} at {time} is CONFIRMED. Please arrive 10 minutes early and pay cash at the clinic. Aapki appointment {clinic_name} mein Dr. {doctor_name} ke paas {date} ko {time} bajay CONFIRM ho gayi hai. Baraye meherbani 10 minute pehle pahunchein. Fees clinic mein cash ada karein.

cancelled (sent on clinic cancel):

> Your appointment at {clinic_name} with Dr. {doctor_name} on {date} at {time} has been cancelled{reason}. Please call the clinic to book again. Aapki appointment {clinic_name} mein Dr. {doctor_name} ke paas {date} ko {time} bajay cancel ho gayi hai{reason}. Dobara book karne ke liye clinic ko call karein.

Phone format: Pakistan mobile `03XX-XXXXXXX` is stored as typed for display, and normalized to `92XXXXXXXXXX` (drop the leading 0, add 92) for the WhatsApp API. Validation regex (after removing spaces/dashes): `^03\d{9}$`.

Sender design: one `sendWhatsApp()` function with two drivers:

- `whatsapp` driver: real WhatsApp Cloud API call (needs env vars).
- `log` driver: used automatically when WhatsApp env vars are missing — writes a `message_log` row with status `not_configured` and never throws, so booking always succeeds. Tests use the `log` driver.

7. Environment variables

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=        # server only
WHATSAPP_TOKEN=                   # server only, Meta WhatsApp Cloud API
WHATSAPP_PHONE_NUMBER_ID=         # server only
WHATSAPP_TEST_MODE=true           # true = log driver fallback allowed
APP_URL=                          # e.g. https://clinicbook.vercel.app
```

8. Security checklist (must all be true at the end)

- [ ] RLS enabled + policies on all 6 tables; verified by attempting cross-clinic reads in tests (must return 0 rows).
- [ ] Anonymous users cannot read `appointments` or `message_log` (test it).
- [ ] Service-role key never imported in client components (grep check).
- [ ] All inputs validated server-side with zod (client validation is UX only).
- [ ] Auth errors are generic; no user-enumeration.
- [ ] No double booking possible: the unique index test (two simultaneous bookings for one slot) results in exactly one success.

- [ ] `npm audit` shows no high/critical vulnerabilities.

9. Performance notes

- Paginate clinic lists (20/page); indexes above cover the hot queries.
- Slot generation is pure date math — no N+1 queries; fetch the day's appointments in one query.
- Target: Lighthouse performance ≥ 85 on the home + booking pages (mobile, throttled 4G).

PART 4 — DESIGN_SPEC.md

ClinicBook — Design Specification (v1)

1. Principles

1. **Phone first, always:** most users are on cheap Android phones. Design for 360px first, desktop second.
2. **Two languages, one screen:** every patient-facing label shows English
   - simple Urdu together (for example: "Book Appointment / اپائنٹمنٹ بک کریں"). Never English-only on patient pages.
3. **One thumb, one action:** one primary button per screen, reachable without stretching.
4. **Every state designed:** loading, empty, error — never a blank screen.
5. **Calm medical feel:** clean, trustworthy, not flashy. Patients are often worried; the design should feel steady.

2. Design tokens **[Decision]**

Token	Value
Fonts	Inter for English (system fallback); Noto Nastaliq Urdu for Urdu text (Google Fonts, free)
Base font size	16px; small 14px; headings 20/24/28px; Urdu text one size larger than its English pair (Nastaliq reads small)
Corner radius	**12px** everywhere (cards, inputs, buttons)
Primary	Teal `#0F766E` (hover `#115E59`) — used for primary buttons + links

Token	Value
Danger	Red `#DC2626` (cancel actions)
Success	Green `#16A34A` (confirmed status)
Warning (pending)	Amber `#D97706` (pending status)
Text	Slate-900 `#0F172A`; secondary Slate-500 `#64748B`
Background	White cards on Slate-50 `#F8FAFC` page
Spacing scale	4 / 8 / 12 / 16 / 24 / 32 px

Dark mode: out of scope for v1.

3. Layout shell

- **Public pages (patients):** top bar with "ClinicBook" + city badge ("Charsadda"), no sidebar. Content max-width 720px, centered — phone-like even on desktop.
- **Clinic pages (staff):** left sidebar on desktop (Dashboard, Doctors, Settings, logout); top bar + slide-over nav on mobile. Content max-width 1100px.
- Urdu text: `dir="rtl"` on Urdu spans only (the page itself stays LTR so layout never breaks).

4. Pages

4.1 Home — clinics list (`/`)

- Header: "Find a doctor in Charsadda / چارسدہ میں ڈاکٹر تلاش کریں".
- Search box (placeholder: "Search clinic or doctor... / کلینک یا ڈاکٹر تلاش کریں...").
- Specialty filter chips: All + the fixed specialty list (General, Children, Women, Skin, Heart, Bones, Eyes, Dental, ENT, Mental, Other).
- Clinic cards: name (bold), address, phone (tap-to-call), doctor count, "View doctors →" button.
- Empty state: "No clinics found. Try another search. / کوئی کلینک نہیں ملا۔ دوبارہ تلاش کریں۔"

4.2 Clinic page (`/clinics/[id]`)

- Clinic name, address, phone (tap-to-call button).
- Doctor cards: name, specialty badge, fee ("Rs {fee} — pay cash at clinic / فیس کلینک میں کیش ادا کریں"), "Book →" button.
- Empty state (no active doctors): "No doctors listed yet. / ابھی کوئی ڈاکٹر درج نہیں ہے۔"

4.3 Doctor + booking page (`/doctors/[id]`)

- Doctor header: name, specialty badge, qualifications, fee note.
- Date strip: next 14 days as tappable day chips (weekday + date).
- Slot grid: time chips (e.g. 4:00pm, 4:30pm); booked slots simply absent (never shown as disabled — absence avoids confusion).
- Day with no schedule: "No clinic today / آج کلینک نہیں ہے".
- Booking form (appears after a slot is picked): name, phone (placeholder `03XX-XXXXXXX`), optional note, big primary button "Confirm booking / بکنگ کنفرم کریں".
- Success screen: checkmark, full summary (doctor, clinic, date, time), "Confirmation sent on WhatsApp / واٹس ایپ پر تصدیق بھیج دی گئی ہے", and a "Book another appointment" link.

4.4 Clinic dashboard (`/dashboard`)

- Header: clinic name + today's date.
- Day filter: date picker + doctor dropdown (All doctors default).
- Count chips: Pending (amber) / Confirmed (green) / Cancelled (slate).
- Appointment rows: time (bold), patient name, phone (tap-to-call), doctor, status badge, Confirm (green) / Cancel (red) buttons for pending rows; cancelled rows show the reason.
- Empty day: "No appointments today. / آج کوئی اپائنٹمنٹ نہیں ہے۔"

4.5 Doctors + schedules (`/doctors`)

- Doctor list rows with Edit / Deactivate buttons.
- "Add doctor" form: name, specialty dropdown, qualifications, fee.
- Schedule editor per doctor: 7 weekday rows, each with a Working toggle, start time, end time, slot-length select (15/30/60).
- Deactivate (not delete) when a doctor has future appointments.

4.6 Settings (`/settings`)

- Clinic profile form: name, city (Charsadda, shown but not editable in v1), address, phone.
- Message log table: date, patient phone (masked except last 4 digits? No — clinic staff see full numbers for their own clinic; that is their own business data), template, status (sent/failed/not_configured), error text if failed.

4.7 Auth (`/login`, `/signup`)

- Centered card (max 400px): email, password, submit; errors shown inline under fields, never as alerts. Signup notes: "One account per clinic. / ایک اکاؤنٹ ایک کلینک کے لیے ہے۔"

5. Components

- `SlotChip`: tappable time chip; selected state = filled teal.
- `StatusBadge`: colored pill per status (Pending=amber, Confirmed=green, Cancelled=slate, Completed=teal outline).
- `BilingualLabel`: renders English + Urdu lines together; used for every patient-facing heading and button.
- `EmptyState`: title (EN + UR) + one-line help + one primary action.
- All interactive elements: visible focus ring (2px teal offset), min 44px touch targets on mobile.

6. States

- **Loading:** skeleton cards (never full-page spinners).
- **Errors:** inline, human language, EN + UR where patient-facing ("Couldn't save — check your connection and try again. / محفوظ نہیں ہو سکا — اپنا انٹرنیٹ چیک کریں اور دوبارہ کوشش کریں۔"), with retry button.
- **Toasts:** bottom-center, auto-dismiss 4s, for confirmations ("Appointment confirmed. WhatsApp sent. / اپائنٹمنٹ کنفرم ہو گئی۔ واٹس ایپ بھیج دیا گیا۔").

7. Responsive + accessibility

- Breakpoints: <640px single column; ≥1024px sidebar layout (clinic side).
- No horizontal scroll on any page at 360px width.
- Contrast: text ≥ 4.5:1. All form inputs have visible labels (no placeholder-only). Keyboard: full booking flow completable without a mouse.

PART 5 — IMPLEMENTATION_PLAN.md

ClinicBook — Implementation Plan (v1)

Build strictly in phase order. Each phase lists its **Definition of Done (DoD)** — a phase is not complete until every DoD item holds.

Phase 0 — Project setup

- Scaffold Next.js 16 (App Router) + TypeScript strict + Tailwind.
- Connect Supabase project; configure `@supabase/ssr`.

- `.env.local` with all vars from TECHNICAL SPEC §7 (document them).
- Lint + format + `tsc --noEmit` wired; Vitest installed.
- **DoD:** `npm run dev` boots; `npm run build` succeeds; env template committed (values never committed).

Phase 1 — Auth + data model

- Supabase Auth: signup (email+password), login, logout, middleware route protection for clinic pages.
- Clinic auto-profile flow: after signup, staff creates their clinic (Charsadda fixed in v1).
- SQL migrations: all tables, indexes (fix the marked defect in TECHNICAL SPEC §3), RLS policies.
- Seed script: city Charsadda + 2 sample clinics + 4 sample doctors with weekly schedules.
- **DoD:** auth flows work end-to-end; RLS proven by a test that clinic A sees zero rows of clinic B's data; anonymous users get zero rows from `appointments` / `message_log`; migrations run cleanly from scratch.

Phase 2 — Public browse

- Home clinics list: search, specialty filter, pagination (20/page).
- Clinic page + doctor cards; empty states per DESIGN SPEC.
- Bilingual labels on all patient pages (EN + simple UR).
- **DoD:** Playwright covers search + filter + clinic page; 360px mobile pass; no horizontal scroll.

Phase 3 — Booking flow + WhatsApp

- Slot generation (TECHNICAL SPEC §5) with unit tests: schedule slicing, booked-slot removal, past-slot removal, cancelled-slot freeing, 14-day window, Asia/Karachi math.
- Doctor page: date strip + slot grid + booking form (zod validation, Pakistan phone format).
- Booking API route: availability re-check in transaction; 409 on taken slot; success screen.
- WhatsApp sender: `whatsapp` driver + `log` driver fallback; message templates from TECHNICAL SPEC §6; `message_log` writes.
- **DoD:** Playwright covers browse → pick slot → book → success screen; double-booking test proves exactly one success for one slot; booking succeeds even when WhatsApp is unconfigured (log driver path).

Phase 4 — Clinic dashboard + management

- Dashboard: today's appointments default, date + doctor filters, count chips, confirm/cancel with WhatsApp triggers, cancel reason.

- Doctors management: add/edit/deactivate; schedule editor (weekday rows, start/end, slot length); schedule changes affect future slots only.
- Settings: clinic profile edit; message log view.
- **DoD:** Playwright covers confirm → WhatsApp logged as sent (log driver) → cancel → WhatsApp logged; schedule edit changes next week's slots but not existing appointments.

Phase 5 — Polish, tests, hardening

- Empty/loading/error states everywhere (per DESIGN SPEC §6).
- Toasts for confirm/cancel actions.
- Full security checklist (TECHNICAL SPEC §8) verified and evidenced.
- Lighthouse ≥ 85 on home + booking pages (mobile, throttled 4G).
- Run the entire `ACCEPTANCE TESTS` suite; fix all failures.
- README updated with run instructions + user-side setup (PART 7).
- **DoD:** every acceptance test passes; no high/critical `npm audit` issues; no `TODO` / `FIXME` left except deferred items.

Explicitly NOT in this plan

Payments, video consultations, lab reports, prescriptions, patient accounts, patient online cancellation, SMS, inbound WhatsApp webhook, admin panel, ratings/reviews, nationwide coverage, mobile apps, dark mode. If the agent proposes any of these, it is wrong — point it back to PRD §5.2.

PART 6 — ACCEPTANCE_TESTS.md

ClinicBook — Acceptance Tests (v1)

Format: **Given / When / Then**. Mark each `[auto]` (Vitest/Playwright) or `[manual]`. The product is done when all pass.

Browse (FR-001, FR-002)

1. [auto] Given seeded Charsadda clinics, when opening home, then only Charsadda clinics show, with name, address, phone, doctor count.
2. [auto] Given a search for "ahmed" (lowercase), when submitted, then all clinics/doctors with "Ahmed" anywhere in the name appear.

3. [auto] Given a specialty filter, when set to Children, then only clinics with a Children doctor appear; filter + search combine (AND logic).
4. [manual] Given a clinic with no active doctors, when opening its page, then the designed empty state shows (not a blank page).

Slots (FR-003)

5. [auto] Given a doctor working Mon 9am–12pm with 30-minute slots, when asking for next Monday's slots, then exactly 6 slots appear.
6. [auto] Given one of those slots is booked (pending), when asking again, then that slot is absent and the other 5 remain.
7. [auto] Given that booking is then cancelled, when asking again, then the slot reappears.
8. [auto] Given today is Monday and it is 11am Asia/Karachi, when asking for today's slots, then slots before 11am are absent.
9. [auto] Given a doctor with no schedule on Friday, when asking for Friday, then zero slots return with the "No clinic today" state.
10. [auto] Given a date 15 days ahead, when asking for slots, then none are offered (14-day window only).

Booking (FR-004)

11. [auto] Given the booking form, when submitted with only name + valid phone `03XX-XXXXXXX`, then the appointment is created with status `pending`.
12. [auto] Given an invalid phone (e.g. `12345`), when submitting, then creation is rejected with a field-level error (both client and API).
13. [auto] Given two simultaneous bookings for the same slot, when both are submitted, then exactly one succeeds and the other gets the 409 "slot was just booked" message.
14. [manual] Given a 360px phone, when completing the whole booking flow, then no horizontal scroll occurs and booking takes under a minute.

WhatsApp (FR-005)

15. [auto] Given a successful booking with WhatsApp unconfigured, when the booking completes, then a `message_log` row exists with status `not_configured` and the booking still succeeded.
16. [auto] Given the log driver, when a clinic confirms an appointment, then a `message_log` row exists with template `confirmed` for the patient's number.
17. [auto] Given a clinic cancel with a reason, when cancelled, then a `message_log` row exists with template `cancelled`.

18. [manual] Given real WhatsApp credentials + approved templates, when a booking is made, then the patient receives the booking-received message on WhatsApp (user-side check, see PART 7).

Clinic auth + isolation (FR-006, FR-007)

19. [auto] Given a wrong password, when logging in, then a generic error shows ("Invalid email or password") with no hint about which part was wrong.
20. [auto] Given clinics A and B, when B queries A's appointment IDs directly, then zero rows are returned (RLS).
21. [auto] Given an anonymous request, when reading `appointments` or `message_log`, then zero rows are returned.
22. [manual] Given the dashboard, when viewed, then today's appointments, count chips, and doctor/date filters are all correct.

Clinic management (FR-008, FR-009)

23. [auto] Given a doctor with a future appointment, when deactivating the doctor, then the doctor disappears from public pages but the appointment remains on the dashboard.
24. [auto] Given a schedule change, when applied, then future slots change but existing appointments keep their original times.
25. [manual] Given Settings, when changing clinic name/address/phone, then the changes persist after reload and show on the public clinic page.
26. [auto] Given a cancelled appointment, when viewing the day list, then it still appears with status `cancelled` (never hard-deleted).

Language + cross-cutting

27. [manual] Given every patient page, when viewed, then every heading and primary button shows English + simple Urdu together.
28. [manual] Given the booking success screen, when viewed, then the fee note states "pay cash at clinic" (no payment UI exists anywhere).
29. [auto] Given `npm audit`, when run at the end, then zero high/critical vulnerabilities.
30. [auto] Given the codebase, when grepped for service-role key imports in client components, then zero matches.
31. [auto] Given the migrations, when run from scratch, then they succeed — proving the marked defect in TECHNICAL SPEC §3 was found and fixed.

PART 7 — VERIFICATION LESSON + SETUP

How the user checks the AI's work (read this — it is the point)

AI coding tools write code fast and confidently, including wrong code. The user's job is not to read every line — it is to run these checks after every phase:

Check 1 — The marked defect. TECHNICAL SPEC §3 contains one deliberately wrong SQL line (an index on the wrong table), clearly marked "INTENTIONAL DEFECT". After Phase 1, ask the tool: *"Did you find the intentional defect in the spec, and what did you change?"* If it says there was no defect, or it copied the wrong line, it walked past the test — make it fix the migration before continuing. This one check tells you more about the tool's care than any demo.

Check 2 — The two prompts that run the loop. Use exactly these, every phase:

1. Build prompt: *"Read PART 1 (AGENTS) first, then build Phase N of PART 5 (IMPLEMENTATION PLAN). Do not skip ahead."*
2. Verify prompt: *"Run the acceptance tests for this phase from PART 6 and fix failures before moving on."* Never let the tool start Phase N+1 while Phase N has failing tests.

Check 3 — Phase-complete proof. A phase is done only when the tool shows: `tsc --noEmit` green, tests green, `npm run build` green, and the phase's acceptance tests quoted as passing. Screenshots of a running page are not proof — test output is proof.

Check 4 — The $0 rule. After the build, ask: *"List every service this app uses and what each one costs."* The only acceptable answer is free tiers (Supabase free, Vercel free, WhatsApp Cloud API free test tier). Any paid service must be removed or approved first.

Check 5 — The scope rule. Ask: *"Did you add anything not in PRD §5.1?"* Payments, video calls, ratings, patient accounts, or SMS are all wrong answers in v1, even if they "look easy".

User-side setup (the builder cannot do these for you)

These steps need the user's own accounts. Nothing here costs money.

1. **Supabase (free tier):** create a project at supabase.com → copy the project URL + anon key + service-role key into `.env.local`. Never share the service-role key with anyone; it never goes in the browser.
2. **WhatsApp Cloud API (free test tier):** create a free Meta developer account → create a WhatsApp app → use the provided test phone number. Note: the test number can only message a small number of verified recipient phone numbers — enough for the experiment, not for a real launch. Submit the three message templates from TECHNICAL SPEC §6 for approval; real sending outside the test setup needs them approved.

3. **Vercel (free tier):** import the project repo → add the same env vars → deploy. The app URL goes in `APP_URL`.
4. **Run the click-through yourself:** browse clinics → book a slot → confirm it as the clinic → check the message log. If any step fails, paste the exact error back to the coding tool.

If this graduates from experiment to real product

Answer these questions first — they are deliberately unanswered in this pack:

1. Would small clinics in Charsadda list themselves, and who onboards them?
2. Would patients trust booking on a new site instead of calling?
3. Would clinics pay later, and how much (monthly fee in PKR)?
4. How does the app reach patients — WhatsApp groups, clinic posters, or something else?
5. What happens when a clinic has no smartphone or no staff to confirm bookings — does the model still work?
6. For selling worldwide later: which languages, timezones, phone formats, and payment methods does each new country need — and does the WhatsApp-first wedge (small clinics taking phone appointments) travel to small clinics in other countries too?

End of ClinicBook PRD Pack v1.0 (experiment). Working title only — rename anytime before launch.